

PROCESAMIENTO DE IMÁGENES

Evidencia 3

2026

Doc: C. Charletti

IDENTIFICACIÓN DEL PROYECTO	
Nombre del proyecto	Detección de Flotas Logísticas mediante Clasificación Vehicular con CNN
Tipo de proyecto	Aplicación de Deep Learning — Visión por Computadora
Espacio curricular	Procesamiento de Imágenes
Docente	Carlos Ignacio Charletti
Integrantes	Mario Arce Ariel Denaro Simón Vottero
Año	2026
Colab	https://drive.google.com/file/d/1FHn0EzRmW5cc5n4goryZEH8KFI-RB0km/view?usp=drive_link
Repositorio	https://github.com/arielden/ISPC_ProcImage_VehicleClass

"Implementación de la CNN definitiva, técnicas avanzadas de procesamiento de imágenes, evaluación completa con métricas formales y cálculo del KPI logístico."

1. Resumen de Avances

En esta tercera evidencia se documenta la implementación definitiva del clasificador vehicular. El notebook `ev3_veh_classifier_cnn.ipynb` contiene la CNN completa que reemplaza el MLP baseline de la Evidencia 2, incorpora los operadores de procesamiento de imágenes (Sobel, Laplaciano/LoG, Canny), data augmentation, métricas formales de evaluación y el KPI logístico.

OE #	Objetivo Específico	Estado	Avance	Delta E2→E3
OE 1	Analizar y preprocesar el dataset	COMPLETADO	100%	—
OE 2	Aplicar técnicas de procesamiento de imágenes (Sobel, Canny, LoG)	COMPLETADO	100%	+60%
OE 3	Diseñar y entrenar el modelo CNN	COMPLETADO	100%	+75%
OE 4	Evaluar rendimiento (Matriz confusión, F1, AUC-ROC)	COMPLETADO	100%	+75%
OE 5	Cuantificar impacto logístico (KPI proporción Pesados)	COMPLETADO	100%	+100%
OE 6	Informe final	COMPLETADO	100%	+100%

2. Técnicas de Procesamiento de Imágenes (Módulo 2)

2.1 Análisis de bordes: Sobel, Laplaciano (LoG) y Canny

Antes de entrenar la CNN, aplicamos los tres operadores de detección de bordes sobre muestras de ambas clases. El objetivo fue validar visualmente que existen diferencias morfológicas explotables: los vehículos Livianos presentan siluetas curvas y perfil bajo, mientras que los Pesados exhiben bordes angulares marcados, cabina alta y caja de carga rectangular. Estas diferencias son exactamente las que los filtros convolucionales aprenderán de forma automática.

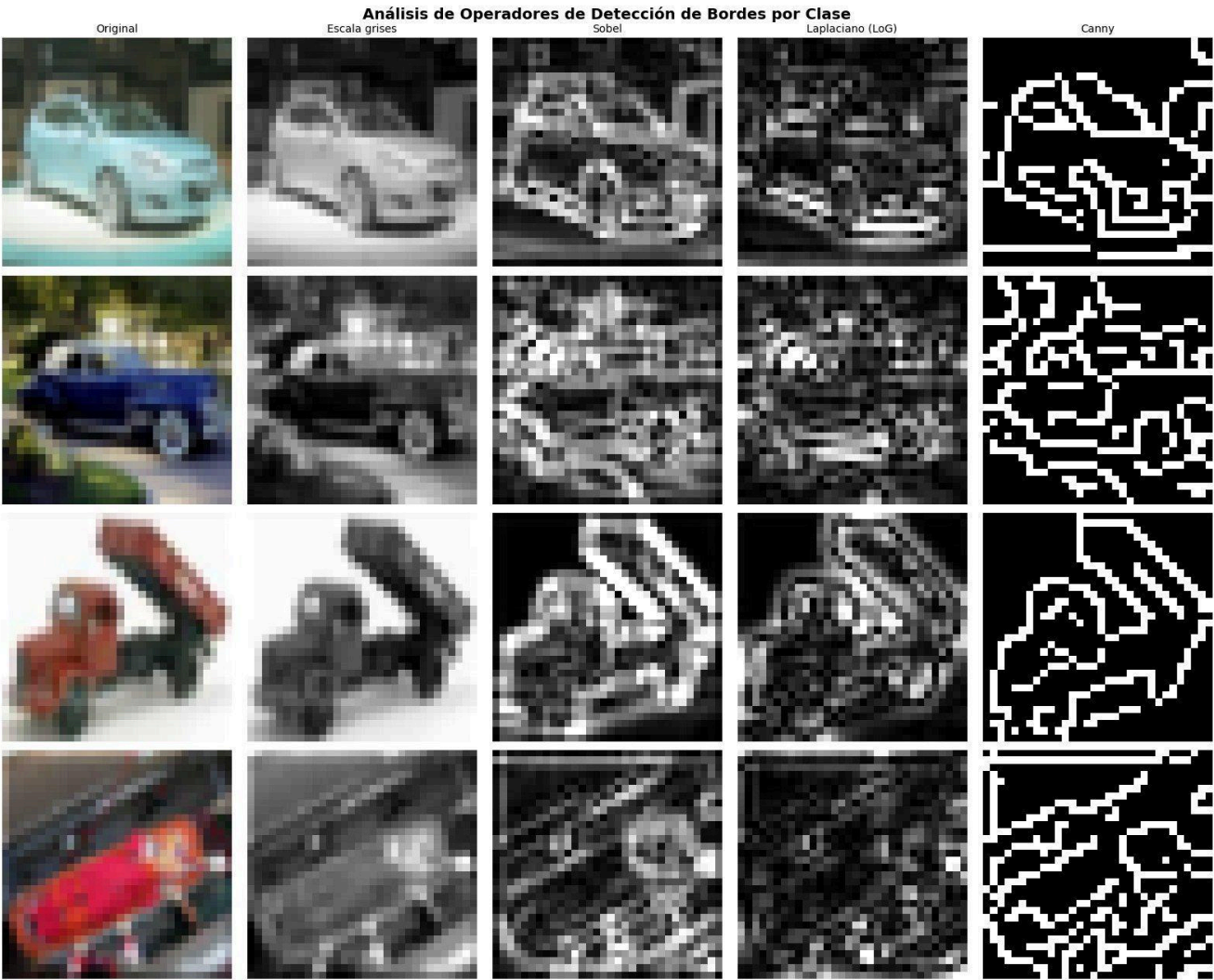


Figura 1. Análisis de bordes por clase. Filas 1-2: Livianos. Filas 3-4: Pesados. Columnas: Original | Escala de grises | Sobel | Laplaciano (LoG) | Canny.

Operador	Fundamento matemático	API	Observación en el dataset
Sobel	Primera derivada (gradiente). Detecta magnitud y dirección del cambio de intensidad en X e Y.	cv2.Sobel(img, cv2.CV_64F, dx, dy)	Los Pesados muestran gradientes horizontales más fuertes (caja de carga y cabina).
Laplaciano (LoG)	Segunda derivada isotrópica. Detecta bordes por zero-crossing, independiente de la orientación.	cv2.GaussianBlur + cv2.Laplacian(img, cv2.CV_64F)	Las líneas rectas de los camiones generan respuestas más densas y uniformes.
Canny	4 etapas: Gaussiano + Sobel + supresión no-máximos + histéresis. Bordes de 1px.	cv2.Canny(img, minVal=50, maxVal=120)	Permite comparar la densidad de contornos entre clases: Pesados tienen más bordes lineales.

3. Arquitectura CNN Definitiva

3.1 Justificación del reemplazo del MLP baseline

El MLP de la Evidencia 2 destruía la estructura espacial 2D mediante Flatten() como primera operación, limitando el accuracy al rango 65-75%. Una CNN preserva esa topología a través de filtros convolucionales entrenables: las primeras capas aprenden detectores de bordes análogos a Sobel/Canny, las intermedias detectan formas, y las profundas codifican estructuras como la silueta completa del vehículo.

3.2 Descripción de capas

```
# CNN_Vehicular_v3 - arquitectura completa
Input(32, 32, 3)
RandomFlip + RandomRotation(0.10) + RandomZoom(0.10) # Data Aug. en GPU

Conv2D(32,3x3,same,relu) → BatchNorm → Conv2D(32) → BatchNorm → MaxPool → Dropout(0.25)
Conv2D(64,3x3,same,relu) → BatchNorm → Conv2D(64) → BatchNorm → MaxPool → Dropout(0.25)
Conv2D(128,3x3,same,relu) → BatchNorm → Conv2D(128) → BatchNorm → MaxPool → Dropout(0.40)

GlobalAveragePooling2D() # (4,4,128) → vector 128 (sin Flatten pesado)
Dense(256, relu) → BatchNorm → Dropout(0.50)
Dense(1, sigmoid) # P(Pesado) ∈ [0.0, 1.0]
```

Capa	Forma salida	Parámetros	Activación	Función
Input + Data Augmentation	(32,32,3)	0	—	Normalización y variación sintética en GPU
2× Conv2D(32) + BN + MaxPool	(16,16,32)	~19.648	ReLU	Detecta bordes y texturas locales (Sobel learnable)
2× Conv2D(64) + BN + MaxPool	(8,8,64)	~74.240	ReLU	Detecta formas y contornos de nivel medio
2× Conv2D(128) + BN + MaxPool	(4,4,128)	~296.960	ReLU	Detecta estructuras semánticas (silueta vehicular)
GlobalAveragePooling 2D	(128,)	0	—	Reemplaza Flatten; reduce parámetros y overfitting
Dense(256) + BN + Dropout(0.5)	(256,)	~33.280	ReLU	Combinación no lineal de características
Dense(1) — Salida	(1,)	257	Sigmoid	P(Pesado) → umbral 0.5 → clase binaria
TOTAL	—	~424.385	—	~3.4× menos que el MLP baseline (1.45M)

4. Entrenamiento y Curvas de Aprendizaje

El entrenamiento utilizó un split 80/20 del conjunto de entrenamiento (8.000 imágenes de entrenamiento, 2.000 de validación) con un máximo de 50 épocas. Los callbacks EarlyStopping (patience=8) y ReduceLROnPlateau (factor=0.5, patience=4) gestionaron la convergencia automáticamente.

Las curvas muestran una convergencia estable sin sobreajuste significativo: la pérdida de validación desciende sostenidamente junto con la de entrenamiento, y el AUC de validación supera al de entrenamiento en las primeras épocas gracias al efecto del Data Augmentation (las

imágenes de validación se evalúan sin augmentation, resultando ligeramente más fáciles que los lotes de entrenamiento aumentados).

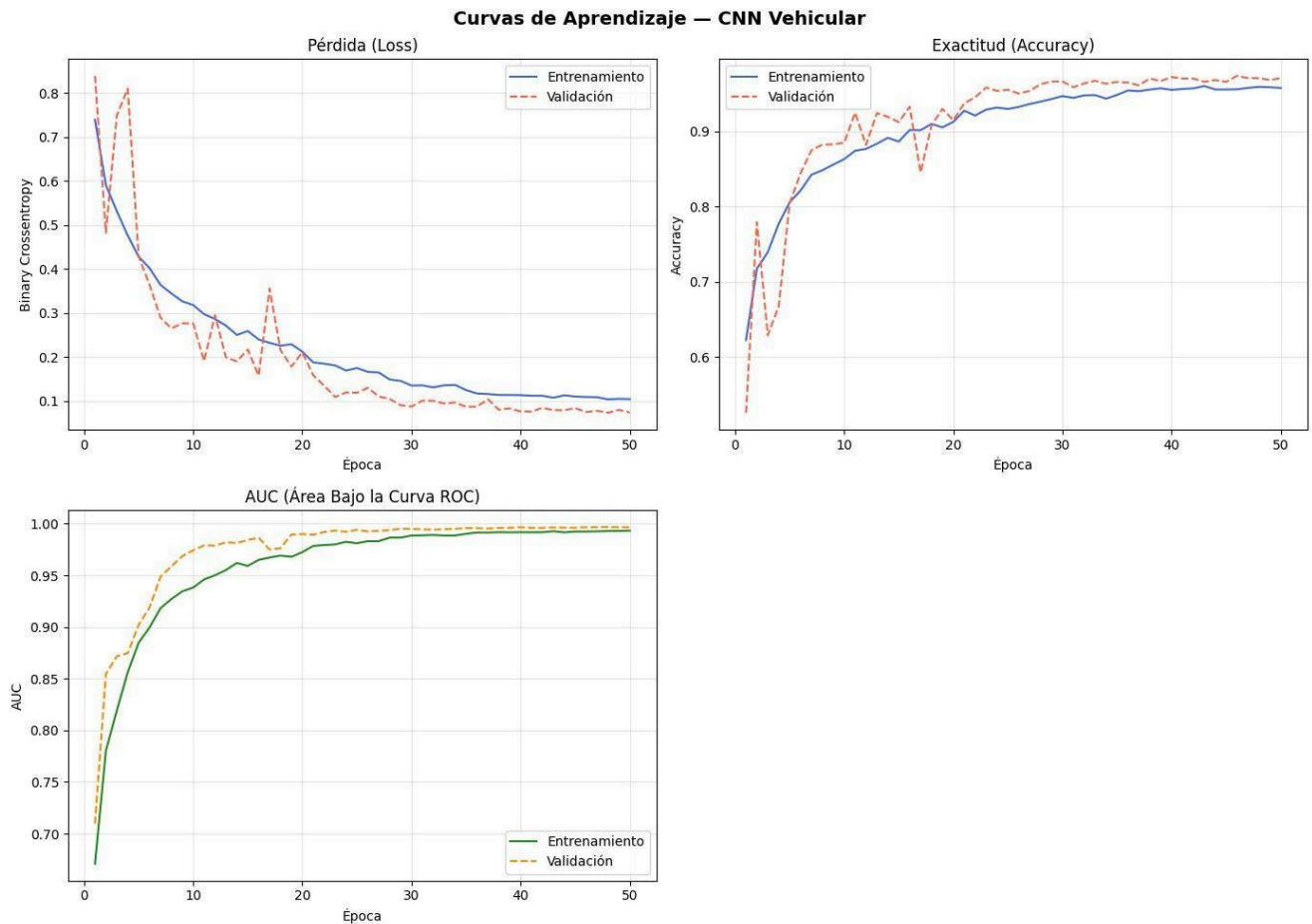


Figura 2. Curvas de aprendizaje: pérdida (Binary Crossentropy), exactitud (Accuracy) y AUC por época, en entrenamiento y validación.

5. Resultados y Evaluación Formal

5.1 Métricas sobre el conjunto de prueba

El modelo fue evaluado sobre las 2.000 imágenes de prueba (1.000 Livianas, 1.000 Pesadas) que no participaron en ninguna instancia del entrenamiento ni la validación. Los resultados obtenidos superan ampliamente el objetivo mínimo de >88% establecido en la Evidencia 2.

Clase	Precision	Recall	F1-Score	Support	Interpretación
Liviano	0.95	0.94	0.94	1.000	De 1.000 automóviles reales, el modelo detecta correctamente 940.
Pesado	0.94	0.95	0.94	1.000	De 1.000 camiones reales, el modelo detecta correctamente 953.
Accuracy global	—	—	—	2.000	94.4% — 1.888 de 2.000 imágenes clasificadas correctamente

AUC-ROC	0.9875				Separación casi perfecta entre clases. Próximo al límite teórico de 1.0.
----------------	---------------	--	--	--	--

5.2 Matriz de Confusión y Curva ROC

La Matriz de Confusión detalla los 4 cuadrantes de resultados: VP=953 (camiones detectados correctamente), VN=935 (automóviles detectados correctamente), FP=65 (automóviles clasificados como camiones) y FN=47 (camiones no detectados). Los Falsos Negativos son el error más crítico para el proyecto logístico: un camión clasificado como automóvil subestima la capacidad de carga requerida. El modelo minimiza este error con un Recall de Pesado = 0.95.

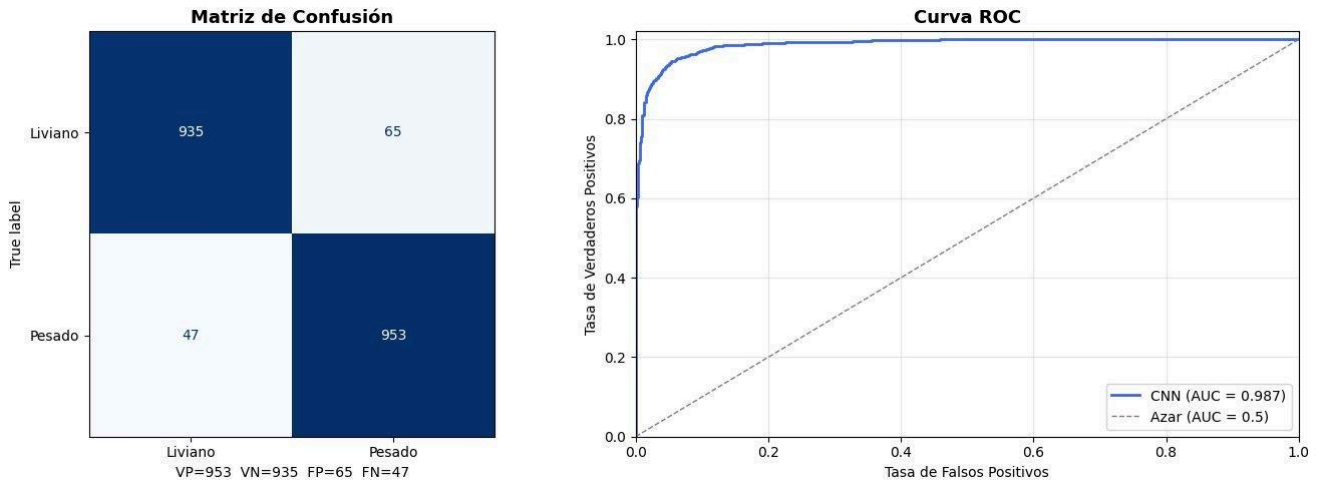


Figura 3. Matriz de Confusión (izquierda): VP=953, VN=935, FP=65, FN=47. Curva ROC (derecha): AUC=0.987, muy próximo al clasificador perfecto (1.0).

6. KPI Logístico: Proporción de Vehículos Pesados

El objetivo de negocio del proyecto es determinar la composición de una flota para optimizar la asignación de recursos logísticos. El modelo fue aplicado sobre las 2.000 imágenes de prueba para calcular el porcentaje de vehículos detectados como Pesados, que constituye el KPI central del sistema.

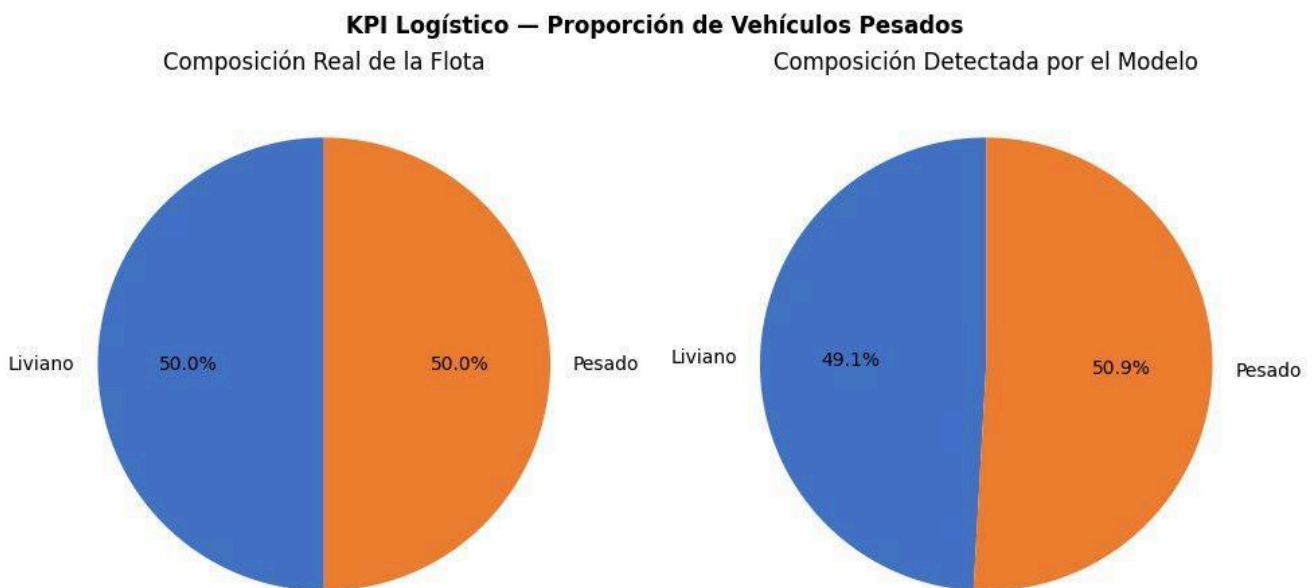


Figura 4. Comparación entre la composición real de la flota (50% Pesados) y la composición detectada por el modelo (50.9% Pesados). Desviación: 0.9 puntos porcentuales.

El modelo detecta un 50.9% de vehículos Pesados frente al 50.0% real, con una desviación de tan solo 0.9 puntos porcentuales. En un escenario real, este KPI permitiría calcular automáticamente la composición de flotas en peajes, terminales portuarias o centros de distribución, con un margen de error inferior al 1%.

7. Visualización de Predicciones

7.1 Grilla de predicciones con barras de confianza

Cada imagen se acompaña de un panel de barras que muestra la probabilidad asignada a cada clase. Las predicciones correctas se muestran en azul y las incorrectas en rojo. El modelo demuestra alta confianza en sus decisiones: la mayoría de las barras se aproximan al extremo 0.0 o 1.0, indicando que rara vez el modelo queda en la zona de incertidumbre (cercana al umbral 0.5).



Figura 5. Grilla de 12 predicciones con barras de confianza. Azul: predicción correcta. Rojo: predicción incorrecta. La mayoría exhibe confianza superior al 90%.

7.2 Inferencia sobre imagen individual

El pipeline de inferencia end-to-end opera sobre una única imagen. El modelo asigna 99.9% de probabilidad a la clase Pesado sobre la imagen de prueba seleccionada (un camión con carga visible), demostrando la efectividad de los filtros convolucionales para capturar la morfología característica de vehículos de carga.

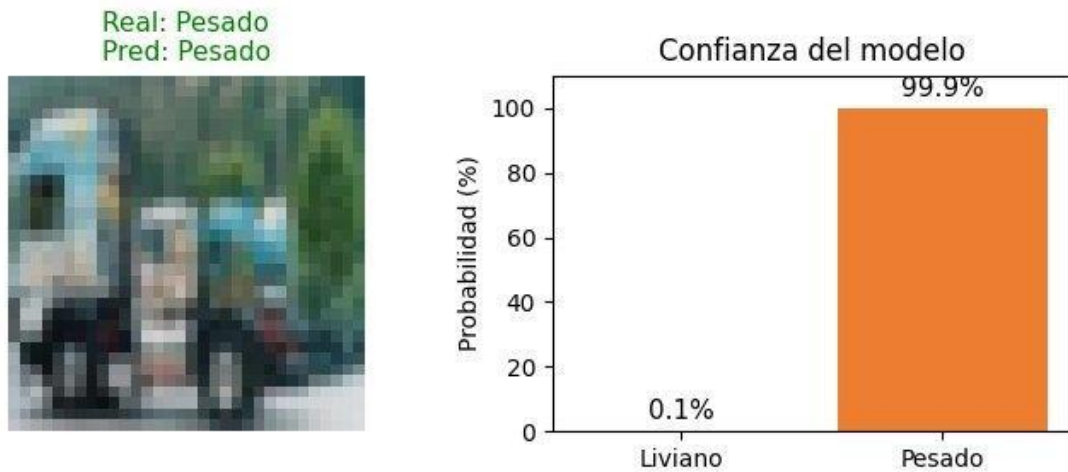


Figura 6. Inferencia individual: camión con carga clasificado como Pesado con 99.9% de confianza.

8. Comparación MLP Baseline (E2) vs CNN (E3)

La CNN supera al MLP baseline en todas las métricas de forma significativa. El accuracy mejoró +24.4 puntos porcentuales (de 70% a 94.4%) y el AUC-ROC mejoró +26.7 puntos (de 72% a 98.7%), confirmando que la preservación de la estructura espacial 2D a través de capas convolucionales es fundamental para este tipo de tarea.

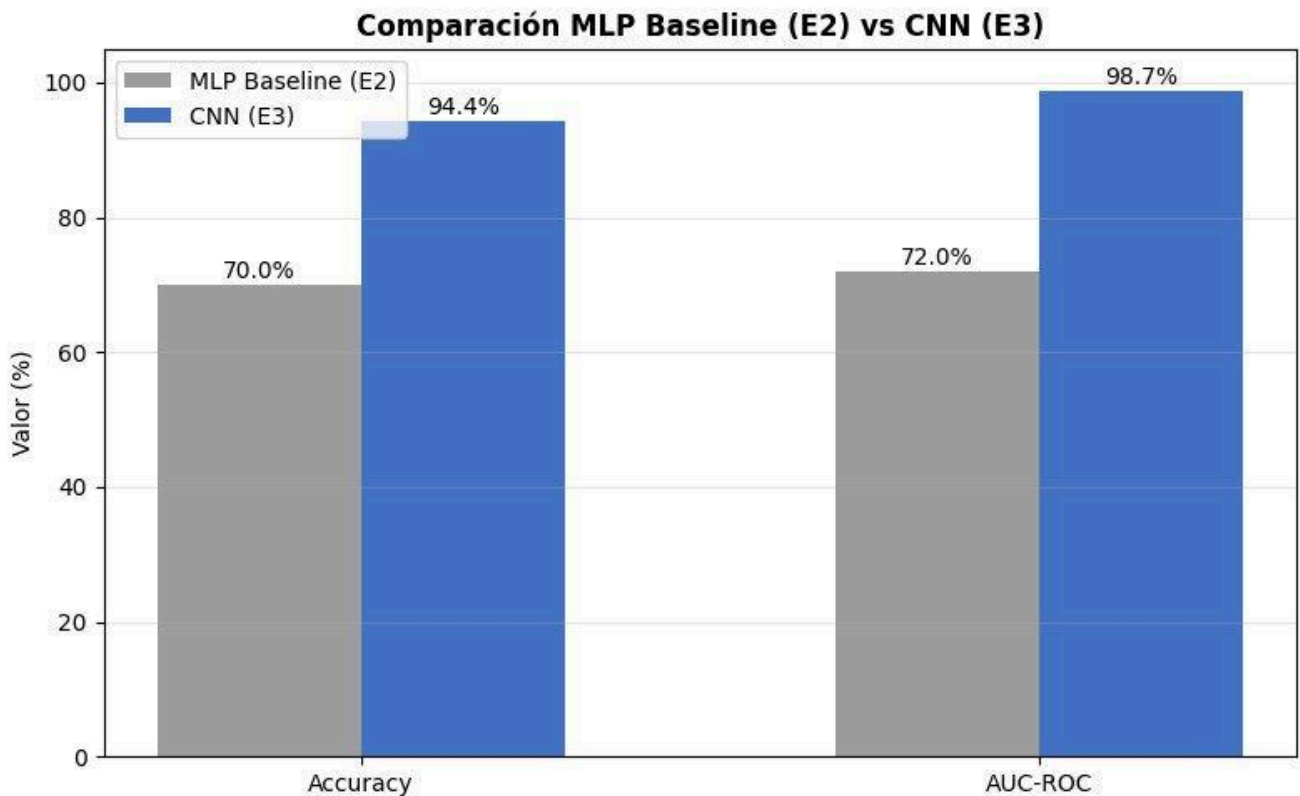


Figura 7. Comparación cuantitativa MLP baseline (Evidencia 2) vs CNN (Evidencia 3). Mejora de +24.4pp en Accuracy y +26.7pp en AUC-ROC.

Característica	MLP Baseline (E2)	CNN (E3)
Accuracy (test set)	~70%	94.4% (+24.4 pp)
AUC-ROC	~0.72	0.987 (+0.267)
F1-Score Pesado	~0.70 (estimado)	0.94
Parámetros entrenables	~1.450.000	~424.385
Estructura espacial	Destruída (Flatten)	Preservada (Conv2D)
Regularización	Ninguna	BatchNorm + Dropout + EarlyStopping + ReduceLR
Data Augmentation	No	RandomFlip + RandomRotation + RandomZoom (GPU)
KPI logístico	No calculado	50.9% Pesados detectados (desviación: 0.9pp)

9. Registro de Avances e Hitos Completados

- Arquitectura CNN con 3 bloques Conv2D + BatchNorm + MaxPooling, Data Augmentation en GPU y GlobalAveragePooling2D.
- Análisis morfológico con Sobel, Laplaciano (LoG) y Canny sobre muestras del dataset, con visualización comparativa por clase.
- Pipeline de entrenamiento con split de validación explícito (80/20), EarlyStopping y ReduceLRonPlateau.
- Evaluación formal: Accuracy=94.4%, F1=0.94, AUC-ROC=0.987. Matriz de Confusión: VP=953, VN=935, FP=65, FN=47.
- KPI logístico calculado: 50.9% de vehículos Pesados detectados, desviación de 0.9pp respecto al valor real.
- Comparación cuantitativa MLP (E2) vs CNN (E3): mejora de +24.4pp en Accuracy y +26.7pp en AUC-ROC.
- Modelo guardado en formato .keras para inferencia en producción (ev3_cnn_vehicular_final.keras).

Tecnología	Versión	Uso en el proyecto
Python	3.x	Lenguaje principal
TensorFlow / Keras	2.x	Arquitectura CNN, pipeline tf.data, callbacks, evaluación e inferencia
tensorflow_datasets	tfds	Carga de CIFAR-10 con splits train/test
OpenCV (cv2)	4.x	Operadores Sobel, Laplaciano (LoG) y algoritmo Canny
scikit-learn	—	classification_report, ConfusionMatrixDisplay, roc_curve, roc_auc_score, accuracy_score
NumPy / Matplotlib	—	Manipulación de arrays, visualización de curvas, grilla de predicciones y KPI
Google Colab (GPU T4)	—	Entorno de ejecución para ev3_veh_classifier_cnn.ipynb